Here is a math problem and a solution of it. Think step by step and verify if the final answer in the solution is correct. The last line of your response should be of the form Answer: $Answer (without quotes) where $Answer is 1 if the final answer in the solution is correct and 0 if incorrect.

Problem
${problem}

Solution
${solution}

Table 1 Prompt template for verification.

3.1 Reinforcement learning for verification

RL Setup. Let $\mathcal{D} = \{(p_i, s_i, y_i)\}_{i=1}^N$ be our dataset, where p_i is a problem, s_i is a solution to the problem, which may be the response from a reasoning model and $y_i \in \{0, 1\}$ represents the correctness of the solution, with 1 indicating correctness and 0 indicating incorrectness. Given a triplet (p_i, s_i, y_i), we prompt a LLM to check the correctness of the solution step-by-step and finally give a conclusion on the correctness, as is shown in Table 1. Denoting the prompt as q_i, the verifier model $\pi_\theta(z_i, y_i'|q_i)$ takes a prompt as input and generates the CoT z_i on judging the correctness of y_i and at last gives a boolean conclusion y if s is correct. The outcome reward function R is as follows:

$$R(y, y') = \begin{cases} 1 & y = y', \\ -1 & y \neq y'. \end{cases}$$

Then the objective of RL is,

$$\mathcal{J}(\theta) = \mathbf{E}_{(q,y)\sim\mathcal{D},(z,y')\sim\pi_\theta(q)} \left[R(y, y')\right]$$

We run the vanilla PPO algorithm on a reasoning model, and propose the following strategy for improvement.

Data collection and filtering. We collect the dataset $\mathcal{D}$ by prompting one or multiple reasoning models to solve problems. For every problem in the dataset, we collect multiple solutions and construct a verification prompt with each solution using the template in Table 1. However, two cases may hurt the RL training, i.e., the extremely difficult problem, which we fail to sample any correct solutions and the extremely easy problems, which we fail to sample any wrong solutions. Such unbalanced data may teach the verifier to be biased on the difficulty of the problem, i.e., be optimistic on easy problems and pessimistic on difficult problems. Therefore, we do not include the data of the two cases in the training dataset.

3.2 Solution selection by Pessimistic Verification

When tackling challenging problems, one can sample multiple solutions and leverage the verifier to identify the most likely correct one. By sampling verification responses multiple times, we can achieve more reliable judgments, thereby improving overall problem-solving performance. We propose a principled and flexible method for the inference time scaling along the two dimensions, i.e., the amount of solutions sampled from the solver model and the amount of verifications sampled from a verifier model. Denote the number of solutions to a problem as N and the number of verifications on each solution is M. We initially conceptualize the selection process as a multi-arm bandit problem, where each arm corresponds to a distinct answer, and each verification constitutes a visit to an arm. The reward is the verification result, which can be either 1 or 0. Each time the solver generates a solution, the arm representing the solution's answer receives M visits and immediately accrues M rewards. The straight-forward approach is to calculate the average reward each arm receives as its score and greedily select the one with the highest score. However, for those with few visits, the score fluctuates

and can be unreliable. Following the pessimism principle in RL, we introduce the lower-confidence-bound, which adds an uncertainty penalty to the score. Let $r_0, r_1, ...r_K$ be the average reward of each answer and $N_0, N_1, ..., N_K$ be the number of solutions that drives to a certain answer. The selection algorithm is defined as:

$$\hat{a} := \arg\max_{a_i} \left(r(a_i) - \alpha \frac{\ln(NM)}{N_i M + 1} \right) \tag{1}$$

where the parameter α is a hyper-parameter that balances the consideration of uncertainty in the decision-making process and ln is the natural logarithm. Intuitively, the first term reflects the signals from the verifier, while the second term accounts for the bias of solver in the answer space.

- When N_i is small, the second term dominates, which neglects the verification and in the extreme case, it collapses to Majority Voting.
- When N_i is large, the first term becomes more important, and in the extreme case, it simply selects the answer with the best verification score.

The phase shift aligns with the fact that Majority Voting is trapped in the bias of the solver, for example when a wrong answer occurs more frequently than the correct one, and as N and M is large, the verification scores stabilize and we tend to trust more on it, because the verification is often easier than the solution.

4 Experiments

4.1 Dataset

Our experiment is on the math problems. The training dataset comes from the AoPS website and official math competition homepages, similar to that of DAPO [28]. We leverage DeepSeek-R1-Distill-Qwen-32B model as the policy model to generate 16 solutions to each problem. We leverage a rule-based program to check if the final answer in the solution is correct, which compares the reference answer of a problem and the answer in the solution and outputs the label, i.e., 1 for a correct response and 0 for the incorrect response. Then we construct the verification dataset with the prompt template in Table 1. To keep the prompt clean and short, we remove the '<think>' part in each solution and only use the summary part.

We test the verification ability on both AIME2024 and AIME2025, 60 questions in total. During training, we monitor the performance on AIME2024 and select the best checkpoint as the final version of Heimdall. Therefore, one can treat AIME2024 as the validation dataset and AIME2025 as the test dataset.

4.2 Scaling of verification

Figure 2 shows the accuracy and the length of response tokens during RL training. As depicted by the blue curve, both accuracy and response length increase with the number of training steps, albeit at different rates. At the early stage, the accuracy improves rapidly, but the response length fluctuates. This is because a minor adjustment to the policy can significantly boost accuracy. Later, the response length grows constantly, while the accuracy gradually converges to 94.5%, because the model is learning to tackle the hardest part in the training dataset, which requires the increasingly more reasoning tokens. The red curve represents the RL training without the data filtering strategy, i.e., incorporating both extreme cases of difficult and easy problems in the dataset. As training progresses, the performance gap becomes more pronounced, indicating that the absence of contrastive examples detrimentally impacts performance. In addition, we test o1-mini with the same evaluation data, which is shown as the dash line in the left of Figure 2. Our model outperforms o1-mini in fewer than 20 steps, indicating substantial potential for enhancing the verification capabilities of general-purpose reasoning models.

We further look into cases to understand what Heimdall has learned during the training. Table 3 shows the verification of a correct solution to a hard problem in AIME2025. Due to space limitations, we only highlight some key points. We can observe two types of checking:

- **Forward checking.** It checks if the reasoning chain in every step of the solution is correct, which is generally applicable to all problems.

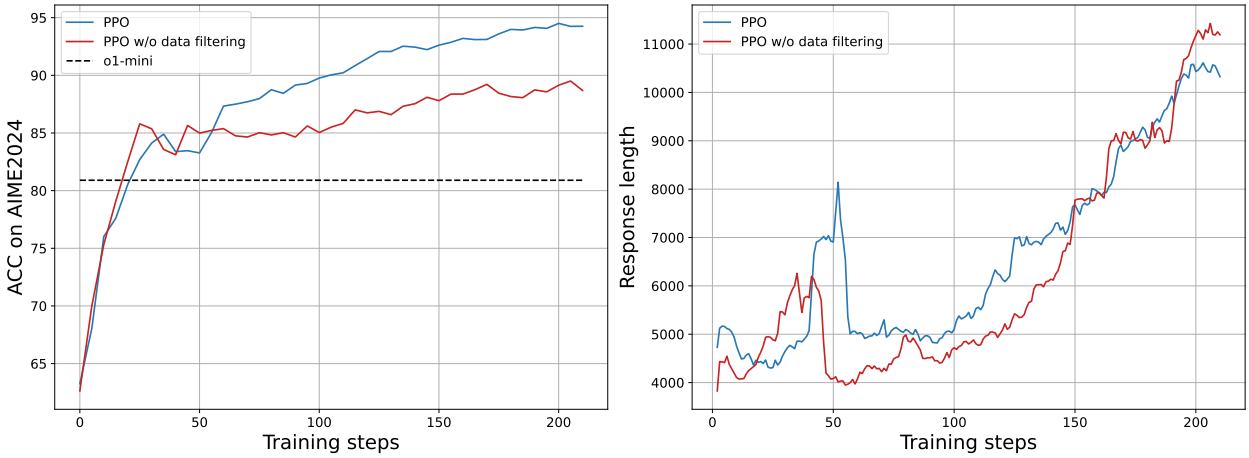

Figure 2 Accuracy and response length during RL training. PPO w/o data filtering is the RL training with all problems in the dataset. **Left:** the accuracy on AIME2024 with the training steps. **Right:** the response length on the training dataset with the training steps.

- **Backward checking.** It checks whether a conclusion, be it intermediate or final, fits the known constraints. For some types of problems like solving equations and finding the general term formula of a sequence, the backward checking is efficient and easy to implement.

The case exemplifies the common task of deriving a general formula for a sequence. As illustrated, Heimdall applies both methods of validation to confirm the correctness of the solution.

Next, we investigate how the verification ability scales as the number of verifications increases. We sample 64 solutions for each problem with the solver model and 64 verifications for each solution, resulting in a total of $30 \times 64 \times 64$ responses on either AIME2024 or AIME2025 dataset. Denoting the number of verifications of each solution as N, we randomly select N verifications for each solution from the data collected above, and determine the final score by some aggregation operation, e.g., Majority Voting and averaging. We repeat the process for 2048 times to eliminate any fluctuations in the statistics. Taking Majority Voting as the aggregation operation, we compute the accuracy, the false positive rate and the false negative rate at every compute budget N, as is shown in the top of Figure 3. In addition, we take the average of N scores, a decimal number in $[0, 1]$, as the final score, and draw the curve of the AUC score in the bottom-left of Figure 3. It shows that Heimdall's performance can be significantly improved by simply repeat sampling more trajectories. As N goes larger, the performance gradually converges to a upper limit. For example, the accuracy converges to about 97.5% on AIME2024 and 96.0% on AIME2025, and the remaining failure cases are the bias inherent in the model that could not be eliminated by adding more compute budget. We further analyze the distribution of those failure cases. For each problem, we calculate two statistics, one is the difficulty of solving it, which is estimated by the pass rate over its 64 solutions. and the other is the difficulty of verifying its solutions, which is estimated by the total number of verification failures on its solutions. Taking the two values as the x-axis and the y-axis respectively, we visualize their correlations in the bottom-right of Figure 3. We observe that the difficulty of a problem does not necessarily correlate to the difficulty of verifying its solutions. For example, Heimdall fails 17/64 times on a problem whose pass-rate is 67%.

In addition, a clear observation is that the performance of Heimdall on AIME2025 is generally worse than that on AIME2024. We believe the main reason is that the verifying solutions on AIME2025 is harder than that on AIME2024. One evidence is that, o1-mini achieves 80.9% in AIME2024 and 75.3% in AIME2025, whose degradation is larger than that of our model.

4.3 Scaling of problem solving with verification

In terms of problem solving, repeated sampling[1] is known to scale with the inference time compute. We evaluate multiple scaling algorithms, including both with and without the verifier. We analyze the inference